

#ASLI ENGINEERING

IO Multiplexing

SWIPE

BY

ARPIT BHAYANI

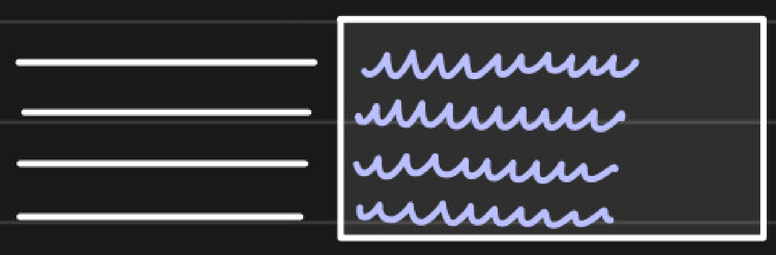
Asynchronous Networking

Supporting Concurrent Clients

Most common way of supporting

Concurrent client is

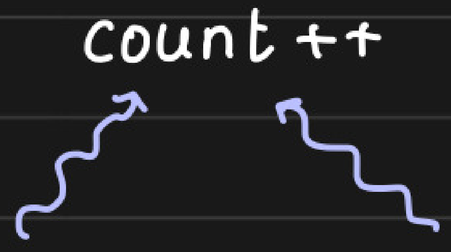
Each client → its own thread



Key Challenge → Making code thread-safe

Threads unnecessarily waits

Slows execution, increases complexity



I/O Multiplexing : Async I/O Programming

All event loops are implemented using this

Python AsyncIO, JavaScript, libevent,

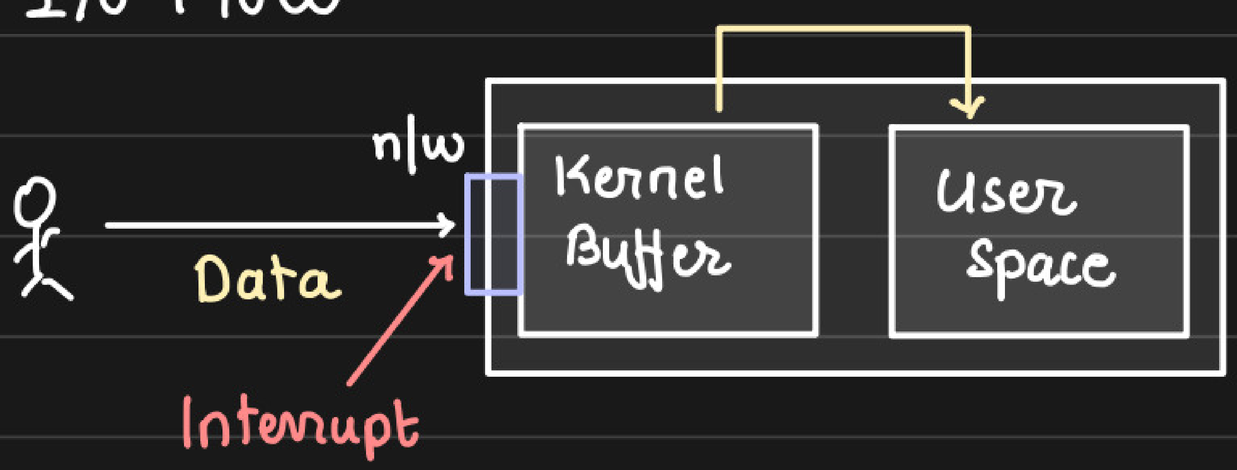
libuv, ... any concurrent single threaded app built with this.

How to actually implement ? EPOLL, KQueue, IOCP

EPOLL monitors a lot of file descriptors for new I/O

↑ we would use this to understand asynchronous programming

I/O Flow



Once in user space, your logic consumes it

ARPIT BHAYANI

Core Idea

what we do

We will continue doing our work,

once a while check if someone is

ready for an I/O ← EPOLL tells us this

if yes, do the I/O

if no, continue

32 bit integer value that uniquely identifies a file - Disk or Socket

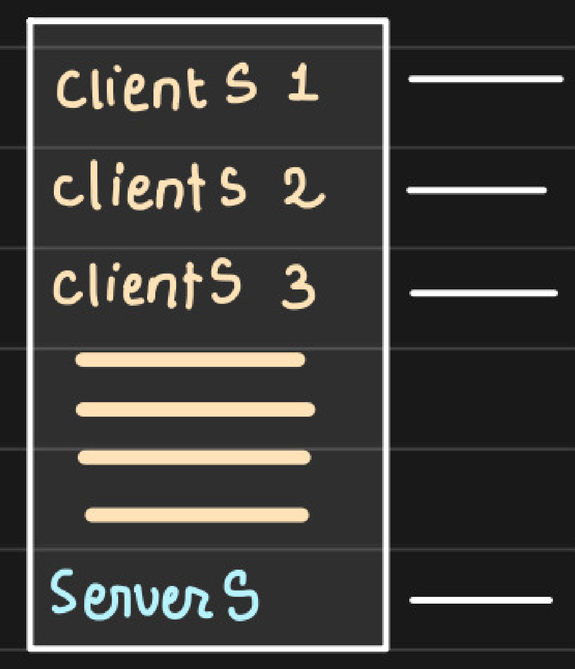
In UNIX everything is a file, hence they have a File Descriptor ↑

Disk I/O | Network I/O

We create a new EPOLLer with epoll_create1

But, how would we know when some I/O is ready?

Hence we ask EPOLL to monitor every single client connection along with the main server socket



we register / deregister File Descriptors with epoll_ctl

↓

Pipes, FIFO, Sockets, inotify, devices, etc

epoll_wait

↳ waits for updates on registered FDs

This is a blocking call, and it moves forward only when some FDs are ready for I/O

ARPIT BHAYANI